

1. Introduction

1. Prérequis & Installation

Installez les dépendances nécessaires dans votre environnement virtuel :

```
pip install sqlalchemy alembic psycopg2-binary python-dotenv
```

Note : On préfère souvent `psycopg2-binary` en développement pour éviter d'avoir à compiler les dépendances C de PostgreSQL localement.

- `sqlalchemy` : L'ORM (Object-Relational Mapper) pour manipuler la BDD avec des objets Python.
- `alembic` : L'outil officiel de suivi et de gestion des versions de schéma de BDD (migrations).
- `psycopg2-binary` : Le driver/pilote de connexion entre Python et PostgreSQL.
- `python-dotenv` : Permet de charger des variables d'environnement depuis un fichier `.env`.

2. Démarrage rapide de la Base de Données (Docker)

Lancez une instance PostgreSQL locale via Docker :

```
docker run --name postgres-dev -p 5432:5432 \  
-e POSTGRES_USER=my_user \  
-e POSTGRES_PASSWORD=my_password \  
-e POSTGRES_DB=my_db \  
-d postgres:16-alpine
```

Créez ensuite un fichier `.env` à la racine de votre projet :

```
DB_URL=postgresql://my_user:my_password@localhost:5432/my_db
```

3. Définition des Modèles avec la syntaxe SQLAlchemy 2.0

SQLAlchemy 2.0 utilise un système moderne basé sur le typage Python (`typing`) avec `Mapped[...]` et `mapped_column()`.

A. La classe de base `DeclarativeBase`

Créez un fichier `database.py` (ou `models.py`) :

```
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    """Classe de base dont hériteront tous nos modèles ORM."""
    pass
```

B. Création des modèles et des relations

Voici un exemple d'application scolaire avec deux tables liées : `Student` et `Course` (relation Un-à-Plusieurs).

```
from datetime import datetime
from typing import List, Optional
from sqlalchemy import String, ForeignKey, func
from sqlalchemy.orm import Mapped, mapped_column, relationship
from database import Base

class Student(Base):
    __tablename__ = "students"

    # Colonnes
    id: Mapped[int] = mapped_column(primary_key=True)
    first_name: Mapped[str] = mapped_column(String(50))
    last_name: Mapped[str] = mapped_column(String(50))
    email: Mapped[str] = mapped_column(String(120), unique=True, index=True)
    # Colonne optionnelle (NULL en BDD)
    bio: Mapped[str | None] = mapped_column(String(255))
    # Date automatique gérée par PostgreSQL
    created_at: Mapped[datetime] = mapped_column(server_default=func.now())
```

Règles importantes de la version 2.0 :

- `Mapped[str]` génère automatiquement un `VARCHAR NOT NULL`.
- `Mapped[str | None]` génère un `VARCHAR NULLABLE`.
- SQLAlchemy déduit le type SQL à partir du type Python (`int` → `INTEGER`, `str` → `VARCHAR`).

4. Configuration d'Alembic (Gestion des Migrations)

A. Initialisation

À la racine de votre projet, exécutez :

```
alembic init alembic
```

Cela crée un dossier `alembic/` et un fichier `alembic.ini`.

B. Configuration de `alembic/env.py`

C'est ici que se trouve le **piège principal** : Alembic doit connaître **la chaîne de connexion** ET **vos modèles** pour pouvoir comparer l'état du code Python avec la base de données.

Modifiez `alembic/env.py` :

```
import os
from logging.config import fileConfig
from dotenv import load_dotenv

from sqlalchemy import engine_from_config, pool
from alembic import context

# 1. Charger les variables d'environnement
load_dotenv()

# Config Alembic classique
config = context.config

# 2. Injecter l'URL du fichier .env dans la config Alembic
config.set_main_option("sqlalchemy.url", os.getenv("DB_URL"))

if config.config_file_name is not None:
    fileConfig(config.config_file_name)

# 3. IMPORTER LA BASE ET LES MODÈLES !
# Attention : Il faut importer TOUS vos modèles ici pour qu'Alembic les détecte.
from database import Base
import models # Assure que tous les modèles sont chargés en mémoire

target_metadata = Base.metadata

# ... garder le reste du fichier env.py tel quel ...
```

5. Workflow de Migration au quotidien

Une fois la configuration terminée, voici la boucle de travail :

Step 1 : Générer la migration automatiquement

À chaque fois que vous modifiez/ajoutez une classe dans vos modèles Python :

```
alembic revision --autogenerate -m "create_students_table"
```

Alembic va comparer vos classes Python avec la base PostgreSQL actuelle et créer un script Python dans `alembic/versions/`.

Step 2 : Vérifier le script généré

Allez toujours jeter un œil dans `alembic/versions/xxxx_create_students_table.py` pour vérifier que les fonctions `upgrade()` et `downgrade()` font bien ce que vous attendez.

Step 3 : Appliquer la migration

Pour répercuter les changements dans PostgreSQL :

```
alembic upgrade head
```

Step 4 (Optionnel) : Revenir en arrière

Si vous vous êtes trompé et voulez annuler la dernière migration :

```
alembic downgrade -1
```

Recap des commandes utiles

Commande	Action
<code>alembic revision --autogenerate -m "msg"</code>	Analyse le code et génère un fichier de migration
<code>alembic upgrade head</code>	Applique toutes les migrations en attente sur la BDD
<code>alembic downgrade -1</code>	Annule la toute dernière migration appliquée
<code>alembic history</code>	Affiche l'historique complet des migrations
<code>alembic current</code>	Affiche le révision_id actuellement appliqué sur la BDD

2 : Colonnes, Contraintes et Relations dans SQLAlchemy 2.0

1. Configurer les Colonnes avec `mapped_column()`

Dans SQLAlchemy 2.0, la gestion de la valeur nulle (`NULL` / `NOT NULL`) est déterminée directement par le type Python via `Mapped[...]` :

- `Mapped[str]` → Génère une colonne `VARCHAR NOT NULL`.
- `Mapped[str|None]` (ou `Mapped[Optional[None]]`) → Génère une colonne `VARCHAR NULLABLE`.

Les règles métiers, contraintes et comportements de la colonne se définissent à l'intérieur de `mapped_column()`.

```
from datetime import datetime
from sqlalchemy import String, Text, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class User(Base):
    __tablename__ = "users"
    # Clé primaire auto-incrémentée
    id: Mapped[int] = mapped_column(primary_key=True)
    # Champ obligatoire avec contrainte d'unicité et un index de recherche
    email: Mapped[str] = mapped_column(String(100), unique=True, index=True)
    # Champ optionnel (nullable)
    bio: Mapped[str|None] = mapped_column(Text)
    # Valeur par défaut côté Python (lors de l'instanciation de l'objet)
    is_active: Mapped[bool] = mapped_column(default=True)
    # Valeur par défaut côté BDD (exécutée directement par PostgreSQL)
    created_at: Mapped[datetime] = mapped_column(server_default=func.now())
```

Tableau récapitulatif des options courantes de `mapped_column()`

Option	Rôle	Exemple
<code>primary_key=True</code>	Définit la colonne comme clé primaire.	<code>mapped_column(primary_key=True)</code>
<code>unique=True</code>	Empêche les doublons dans cette colonne.	<code>mapped_column(unique=True)</code>
<code>index=True</code>	Crée un index BDD pour accélérer les recherches.	<code>mapped_column(index=True)</code>
<code>default=...</code>	Valeur par défaut attribuée par Python .	<code>mapped_column(default="student")</code>
<code>server_default=...</code>	Valeur par défaut générée par le SGBD (ex: <code>func.now()</code>).	<code>mapped_column(server_default=func.now())</code>

default vs server_default :

- `default` est évalué par Python au moment où tu crées l'instance `User()`.
- `server_default` charge le moteur de la base de données de générer la valeur (très utile pour `NOW()`, les séquences ou les UUIDs natifs).

2. Définir des Relations entre Tables

Pour lier deux tables (ou une table avec elle-même), on associe systématiquement deux éléments :

1. **La contrainte physique en BDD** : `ForeignKey("nom_table.colonne")` (utilise le nom de la table SQL).
 2. **La navigation en Python** : `relationship("NomClasse", ...)` (utilise le nom de la classe Python).
-

A. Relation Un-à-Plusieurs (1-N) & Plusieurs-à-Un (N-1)

C'est la relation la plus courante. Un parent possède plusieurs enfants (ex: Un enseignant a plusieurs cours).

```
from typing import List
from sqlalchemy import String, ForeignKey
from sqlalchemy.orm import Mapped, mapped_column, relationship

class Teacher(Base):
    __tablename__ = "teachers"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50))

    # Côté 1 : Liste d'objets Course
    # cascade="all, delete-orphan" supprime les cours associés si le prof est
    supprimé
    courses: Mapped[List["Course"]] = relationship(
        back_populates="teacher",
        cascade="all, delete-orphan"
    )

class Course(Base):
    __tablename__ = "courses"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(100))

    # Côté N : Clé étrangère physique
    teacher_id: Mapped[int] = mapped_column(ForeignKey("teachers.id"))

    # Côté N : Objet Teacher unique
    teacher: Mapped["Teacher"] = relationship(back_populates="courses")
```


B. Relation Un-à-Un (1-1)

Chaque enregistrement d'une table est lié à un seul enregistrement d'une autre table (ex: Un utilisateur a un seul profil).

Dans SQLAlchemy 2.0, on utilise simplement une annotation `Mapped["Profile"]` (sans `List`) des deux côtés, et une contrainte `unique=True` sur la clé étrangère.

```
class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(String(50))

    # Côté 1 : Un seul profil (pas de liste)
    profile: Mapped[Optional["Profile"]] = relationship(
        back_populates="user",
        cascade="all, delete-orphan"
    )

class Profile(Base):
    __tablename__ = "profiles"

    id: Mapped[int] = mapped_column(primary_key=True)
    bio: Mapped[str] = mapped_column(String(255))

    # Clé étrangère UNIQUE pour garantir la contrainte 1-1
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), unique=True)

    user: Mapped["User"] = relationship(back_populates="profile")
```

C. Relation Plusieurs-à-Plusieurs (N-N / Many-to-Many)

Un étudiant peut s'inscrire à plusieurs cours, et un cours rassemble plusieurs étudiants. Cela requiert une **table d'association** intermédiaire.

Option 1 : Table d'association simple (sans données supplémentaires)

```
from sqlalchemy import Table, Column

# Table de liaison gérée automatiquement
student_course = Table(
    "student_course",
    Base.metadata,
    Column("student_id", ForeignKey("students.id"), primary_key=True),
    Column("course_id", ForeignKey("courses.id"), primary_key=True),
)

class Student(Base):
    __tablename__ = "students"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50))

    # Relation N-N désignée par secondary
    courses: Mapped[List["Course"]] = relationship(
        secondary=student_course,
        back_populates="students"
    )

class Course(Base):
    __tablename__ = "courses"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(100))

    students: Mapped[List["Student"]] = relationship(
        secondary=student_course,
        back_populates="courses"
    )
```

Option 2 : Table d'association avec attributs (ex: Note, Date d'inscription)

Si la relation stocke des informations propres à la liaison, on crée un **modèle ORM dédié**.

```
class Enrollment(Base):
    __tablename__ = "enrollments"

    student_id: Mapped[int] = mapped_column(ForeignKey("students.id"),
primary_key=True)
    course_id: Mapped[int] = mapped_column(ForeignKey("courses.id"),
primary_key=True)

    # Information propre à l'inscription
    grade: Mapped[Optional[float]]

    student: Mapped["Student"] = relationship(back_populates="enrollments")
    course: Mapped["Course"] = relationship(back_populates="enrollments")

class Student(Base):
    __tablename__ = "students"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50))

    enrollments: Mapped[List["Enrollment"]] =
relationship(back_populates="student")

class Course(Base):
    __tablename__ = "courses"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(100))

    enrollments: Mapped[List["Enrollment"]] =
relationship(back_populates="course")
```

D. Relations Récursives / Auto-référencées (Self-Referencing)

Une table fait référence à elle-même (ex: Arborescence de catégories, Hiérarchie d'entreprise).

1. Hiérarchie Simple (1-N Récursif : Employé / Manager)

```
class Employee(Base):
    __tablename__ = "employees"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50))

    # Clé étrangère pointant vers l'id de la MÊME table
    manager_id: Mapped[Optional[int]] = mapped_column(ForeignKey("employees.id"))

    # Relation vers le manager (Parent)
    manager: Mapped[Optional["Employee"]] = relationship(
        back_populates="subordinates",
        remote_side=[id] # Indique quel champ représente le côté parent
    )

    # Relation vers les subordonnés (Enfants)
    subordinates: Mapped[List["Employee"]] = relationship(
        back_populates="manager"
    )
```

Le paramètre `remote_side=[id]` : C'est le point fondamental des relations récursives. Il indique à SQLAlchemy que le champ `id` pointe vers le parent.

3. Synthèse des Relations pour les Élèves

Type de Relation	Type Python Côté A	Type Python Côté B	Clé Étrangère / Propriété clé
Un-à-Plusieurs (1-N)	<code>Mapped[List["B"]]</code>	<code>Mapped["A"]</code>	<code>ForeignKey("a.id")</code> sur B
Un-à-Un (1-1)	<code>Mapped[Optional["B"]]</code>	<code>Mapped["A"]</code>	<code>ForeignKey("a.id", unique=True)</code> sur B
Plusieurs-à-Plusieurs (N-N)	<code>Mapped[List["B"]]</code>	<code>Mapped[List["A"]]</code>	Table intermédiaire via <code>secondary=...</code>
Récursif (1-N)	<code>Mapped[List["Node"]]</code>	<code>Mapped[Optional["Node"]]</code>	<code>remote_side=[id]</code> côté parent

3 : L'Engine, la Session et l'Exécution des Requêtes

Après avoir défini nos modèles ORM et appliqué nos migrations avec Alembic, il est temps d'interagir avec la base de données.

En **SQLAlchemy 2.0**, toutes les opérations passent par deux objets fondamentaux : l'**Engine** (le moteur de connexion) et la **Session** (le gestionnaire de transactions).

1. Connexion et Création de la Session

A. L'Engine (**create_engine**)

L'**Engine** est le composant de bas niveau. Il maintient un **pool de connexions** vers la BDD et traduit le code Python en SQL natif selon le SGBD (PostgreSQL, SQLite, etc.).

B. Le **sessionmaker** et le pattern Generator

Une **Session** représente une transaction unique (une "unité de travail"). Elle conserve en mémoire tampon les objets Python chargés ou modifiés.

Voici la manière idiomatique et sécurisée d'exposer la session (très courante dans les scripts, CLI ou frameworks web comme FastAPI) :

```
import os
from typing import Generator
from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker

load_dotenv()

# 1. Le Moteur de connexion (echo=True affiche les requêtes SQL générées)
engine = create_engine(
    url=os.getenv("DB_URL", ""),
    echo=True
)

# 2. La fabrique de sessions
session_maker = sessionmaker(bind=engine)

# 3. Le générateur de Session avec gestion automatique de la transaction
def get_session() -> Generator[Session, None, None]:
    session = session_maker()
    try:
        yield session
        session.commit() # Valide la transaction si aucune erreur
    except Exception:
        session.rollback() # Annule la transaction en cas d'erreur
        raise
    finally:
```

```
session.close()      # Libère la connexion et la remet dans le pool
```

Le rôle du `yield` :

- Le code qui utilise la session s'exécute pendant le `yield`.
- Si tout se passe bien, le code reprend après le `yield` et fait un `commit()`.
- Si une exception survient pendant l'utilisation, le bloc `except` intercepte l'erreur, fait un `rollback()` puis relance l'exception.
- Le `finally` garantit que la session est **toujours fermée** (`close()`), évitant les fuites de connexions.

Alternative avec Context Manager (`@contextmanager`)

Si tu veux utiliser cette fonction avec la syntaxe `with`, on utilise le décorateur `@contextmanager` de la bibliothèque standard :

```
from contextlib import contextmanager

@contextmanager
def get_session():
    session = session_maker()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()

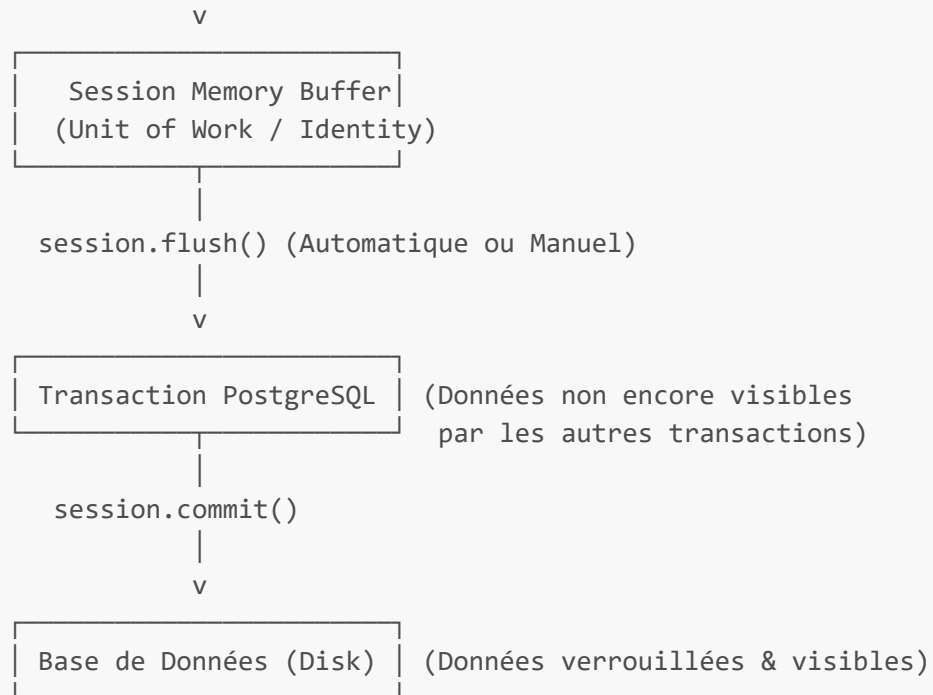
# Utilisation simple :
with get_session() as session:
    # tes requêtes ici...
    pass # Le commit est automatique à la sortie du 'with'
```

2. Le cycle de vie d'une Session : `flush()`, `commit()`, `rollback()` et `close()`

Comprendre ce qui se passe sous le capot évite 90 % des bugs lors des requêtes.

Instance Python

| `session.add(obj)`



1. `flush()` : Envoyer les ordres SQL

- **Ce qu'il fait** : Il traduit les objets modifiés/ajoutés en requêtes SQL (`INSERT`, `UPDATE`, `DELETE`) et les envoie à la base de données **à l'intérieur de la transaction courante**.
- **Ce qu'il ne fait pas** : Il **ne valide pas** la transaction définitivement.
- **Quand l'utiliser ?** Quand tu as besoin de récupérer un identifiant généré par la BDD (ex: un `id` auto-incrémenté ou un `created_at` généré par le SGBD) **avant** de valider définitivement la transaction.

```

user = User(name="Alice")
session.add(user)

print(user.id) # None (L'id n'existe pas encore côté Python)

session.flush() # Envoie l'INSERT à la BDD. La BDD génère l'id.

print(user.id) # 1 (L'id est maintenant disponible !)
```

Autoflush : Par défaut, SQLAlchemy exécute un `flush()` automatique juste avant chaque requête de lecture (`select`), pour s'assurer que tes requêtes lisent des données à jour.

2. `commit()` : Valider la transaction

- Persiste de manière **permanente** les changements en BDD.
- Exécute un `flush()` sous le capot si des données n'avaient pas encore été envoyées.
- Libère les verrous (*locks*) posés sur les tables/lignes.
- **Important** : Après un `commit()`, par défaut, SQLAlchemy passe tous les objets de la session dans un état "expiré" (*expired*). La prochaine fois que tu liras une propriété d'un objet (`user.name`), SQLAlchemy

refaira silencieusement une requête **SELECT** pour recharger la donnée fraîche depuis la BDD.

3. `rollback()` : Annuler la transaction

- Annule toutes les requêtes SQL envoyées depuis le début de la transaction ou depuis le dernier `commit()`.
- Remet la session dans un état propre.
- **Obligatoire** en cas d'erreur BDD (ex: violation de contrainte d'unicité) sous peine de bloquer les futures requêtes sur la même session.

4. `close()` : Libérer la session

- Ferme la session ORM et remet la connexion réseau dans le pool de connexions de l'Engine.
- Détache tous les objets Python de la session (ils deviennent des objets déconnectés ou *detached*).

3. Exécuter des requêtes : `execute()` et les méthodes de résultat

Dans **SQLAlchemy 2.0**, l'ancienne syntaxe `session.query(User)` est abandonnée au profit de `session.execute()` associée à la fonction `select()`.

```
from sqlalchemy import select

# 1. Construire la déclaration (Statement)
stmt = select(User).where(User.email == "alice@example.com")

# 2. Exécuter la déclaration
result = session.execute(stmt)
```

L'objet `result` retourné par `session.execute()` est un itérable de type `Result`. Voici les méthodes essentielles pour extraire tes données :

Tableau des méthodes d'extraction de résultats

Méthode	Description	Cas d'usage principal
<code>scalars().all()</code>	Renvoie une liste d'objets ORM directement.	Récupérer plusieurs objets (<code>List[User]</code>).
<code>scalars().first()</code>	Renvoie le premier objet ou <code>None</code> si aucun résultat.	Recherche optionnelle d'un seul élément.
<code>scalar_one()</code>	Renvoie exactement un objet . Lève une exception si 0 ou >1 résultat.	Recherche par ID ou clé unique stricte.
<code>scalar_one_or_none()</code>	Renvoie un objet ou <code>None</code> . Lève une exception si >1 résultat.	Recherche par email / slug / unique.

Méthode	Description	Cas d'usage principal
<code>scalars().one()</code>	Identique à <code>scalar_one()</code> .	Recherche stricte d'un objet ORM.

Exemples d'utilisation rapide

```
# A. Récupérer tous les utilisateurs actifs
stmt = select(User).where(User.is_active == True)
users: list[User] = list(session.scalars(stmt).all())

# B. Récupérer un utilisateur par son ID (Raccourci pratique)
user = session.get(User, 1) # Utilise la clé primaire directement (plus rapide)

# C. Récupérer par email (Unique)
stmt = select(User).where(User.email == "alice@example.com")
user = session.scalar(stmt) # Raccourci équivalent à
session.execute(stmt).scalar_one_or_none()
```

Remarque sur `session.scalars(...)` : C'est un raccourci très pratique en SQLAlchemy 2.0 ! Écrire `session.scalars(stmt)` équivaut exactement à écrire `session.execute(stmt).scalars()`.

4. Le CRUD Complet en SQLAlchemy 2.0

Le CRUD représente les 4 opérations fondamentales pour interagir avec une base de données.

Contrairement aux requêtes SQL brutes, l'ORM SQLAlchemy vous permet de manipuler directement des objets Python tout en traduisant vos actions en requêtes SQL optimisées.

A. CREATE (Insertion de données)

Pour insérer de nouvelles lignes en base de données, on instancie un objet Python puis on l'ajoute à la session avec `session.add()` (ou `session.add_all()`).

1. Insertion simple (Un seul enregistrement)

```
# 1. Instanciation de l'objet Python
new_teacher = Teacher(
    name="Sarah Connor",
    email="s.connor@school.com"
)

# 2. Ajout à la session (l'objet passe dans l'état 'pending')
session.add(new_teacher)
```

```
# 3. Validation de la transaction (INSERT physique en BDD)
session.commit()

# L'ID généré par la BDD est désormais accessible
print(f"Enseignant créé avec l'ID : {new_teacher.id}")
```

2. Insertion multiple (add_all)

```
teachers = [
    Teacher(name="Alan Turing", email="a.turing@school.com"),
    Teacher(name="Ada Lovelace", email="a.lovelace@school.com")
]

session.add_all(teachers)
session.commit()
```

3. Insertion avec relations (Cascade automatique)

Si vos modèles définissent une relation, vous pouvez insérer un objet parent et ses enfants en **une seule opération** :

```
# Création du prof avec ses cours directement
prof = Teacher(
    name="Guido van Rossum",
    email="guido@python.org",
    courses=[
        Course(title="Python 101"),
        Course(title="SQLAlchemy 2.0")
    ]
)

# Un seul add() suffit ! SQLAlchemy va ajouter le prof ET les 2 cours
session.add(prof)
session.commit()
```

B. READ (Lecture et filtrage)

Dans SQLAlchemy 2.0, toutes les lectures s'effectuent en construisant une instruction avec `select()` puis en l'exécutant sur la session.

1. Récupérer par Clé Primaire (session.get)

C'est l'un des rares cas où il n'est pas nécessaire d'écrire un `select()`. C'est ultra rapide car SQLAlchemy cherche d'abord dans le cache interne de la session avant d'interroger la BDD.

```
# Récupère le Teacher avec l'id 1 (renvoie None si introuvable)
teacher = session.get(Teacher, 1)

if teacher:
    print(teacher.name)
```

2. Récupérer tous les éléments (`all`)

```
from sqlalchemy import select

stmt = select(Teacher)
teachers = session.scalars(stmt).all() # Renvoie une list[Teacher]

for t in teachers:
    print(t.name)
```

3. Filtrer les résultats (`where`)

On passe des conditions booléennes à la méthode `.where()` :

```
# Égalité
stmt = select(Teacher).where(Teacher.email == "guido@python.org")
teacher = session.scalar(stmt) # Récupère un seul résultat (ou None)

# Conditions multiples (AND automatique)
stmt = select(Course).where(
    Course.teacher_id == 1,
    Course.title.ilike("%python%") # CASE-INSENSITIVE LIKE
)
courses = session.scalars(stmt).all()
```

4. Opérateurs courants pour `.where()`

Opérateur SQL	Syntaxe SQLAlchemy	Exemple
<code>=</code>	<code>==</code>	<code>Teacher.name == "Alice"</code>
<code>!=</code>	<code>!=</code>	<code>Teacher.name != "Bob"</code>
<code>LIKE / ILIKE</code>	<code>.like() / .ilike()</code>	<code>Teacher.email.ilike("%@gmail.com")</code>

Opérateur SQL	Syntaxe SQLAlchemy	Exemple
IN	<code>.in_([])</code>	<code>Teacher.id.in_([1, 2, 5])</code>
IS NULL	<code>== None</code> ou <code>.is_(None)</code>	<code>Course.description.is_(None)</code>
AND	<code>,</code> ou <code>and_()</code>	<code>where(Cond1, Cond2)</code>
OR	<code>or_()</code>	<code>where(or_(Cond1, Cond2))</code>

Exemple avec **OR** et **IN** :

```
from sqlalchemy import or_

stmt = select(Course).where(
    or_(
        Course.id.in_([10, 20, 30]),
        Course.title == "FastAPI"
    )
)
results = session.scalars(stmt).all()
```

5. Tri, Limite et Pagination

```
# ORDER BY, LIMIT, OFFSET
stmt = (
    select(Course)
    .where(Course.teacher_id == 1)
    .order_by(Course.title.asc()) # Ou desc()
    .offset(0)                    # Sauter N résultats
    .limit(10)                   # Récupérer N résultats
)

courses = session.scalars(stmt).all()
```

C. UPDATE (Modification de données)

Il existe deux manières de mettre à jour des données en SQLAlchemy.

Méthode 1 : Modification via l'ORM (Recommandée)

C'est la méthode naturelle. Vous chargez un objet en mémoire, vous modifiez ses attributs Python, puis vous faites un `commit()`. SQLAlchemy détecte automatiquement les changements (*Unit of Work*).

```
# 1. Charger l'objet
teacher = session.get(Teacher, 1)

if teacher:
    # 2. Modifier les propriétés
    teacher.name = "Sarah Connor-Reese"
    teacher.email = "s.reese@school.com"

    # 3. Commit (SQLAlchemy génère automatiquement un UPDATE SQL)
    session.commit()
```

Pas besoin de refaire un `session.add(teacher)` ! Tant que l'objet est attaché à la session, SQLAlchemy suit ses modifications.

Méthode 2 : UPDATE en masse (`update()` direct)

Si vous voulez mettre à jour des dizaines ou des centaines de lignes sans avoir à charger chaque objet en mémoire Python, utilisez l'instruction `update()` :

```
from sqlalchemy import update

# UPDATE courses SET title = 'DRAFT - ' || title WHERE teacher_id = 5
stmt = (
    update(Course)
    .where(Course.teacher_id == 5)
    .values(title="DRAFT - " + Course.title)
)

session.execute(stmt)
session.commit()
```

D. DELETE (Suppression)

Tout comme pour le `UPDATE`, il existe deux manières de supprimer des données.

Méthode 1 : Suppression via l'ORM (Recommandée)

```
# 1. Récupérer l'objet
course = session.get(Course, 10)

if course:
    # 2. Marquer l'objet comme supprimé
    session.delete(course)
```

```
# 3. Validation de la suppression en BDD
session.commit()
```

Comportement avec les cascades : Si vous avez configuré `cascade="all, delete-orphan"` dans votre relation `Teacher.courses`, supprimer un `Teacher` avec `session.delete(teacher)` supprimera **automatiquement tous ses cours associés** en BDD.

Méthode 2 : DELETE en masse (`delete()` direct)

Utile pour vider ou nettoyer des données en masse sans charger les objets :

```
from sqlalchemy import delete

# DELETE FROM courses WHERE created_at < '2023-01-01'
stmt = delete(Course).where(Course.created_at < "2023-01-01")

session.execute(stmt)
session.commit()
```

Récapitulatif du Workflow CRUD

```
CREATE → instancier -> session.add(obj)      -> session.commit()
READ  → stmt = select(...)                    -> session.scalars(stmt)
UPDATE → obj = session.get(...) -> obj.attr = "..." -> session.commit()
DELETE → obj = session.get(...) -> session.delete(obj) -> session.commit()
```

5 : Requêtes Avancées, Chargement des Relations et Jointures

Jusqu'à présent, nous avons manipulé des modèles isolés. Mais en pratique, la vraie puissance d'un ORM réside dans sa capacité à naviguer à travers les relations.

Cependant, mal gérer la récupération des relations peut rapidement effondrer les performances de ton application.

1. Le Lazy Loading et le Danger du Piège "N+1 Select"

A. Qu'est-ce que le Lazy Loading ?

Par défaut dans SQLAlchemy, les relations sont configurées en **Lazy Loading** (*chargement paresseux*).

Cela signifie que lorsque tu récupères un objet (ex: un **Teacher**), SQLAlchemy **ne charge pas** ses relations (ex: ses **courses**) immédiatement. La requête SQL pour récupérer les cours n'est exécutée **qu'au moment exact où tu accèdes à la propriété `teacher.courses`** dans ton code Python.

```
# 1. SQLAlchemy exécute : SELECT * FROM teachers WHERE id = 1;
teacher = session.get(Teacher, 1)

# 2. C'est ICI que SQLAlchemy déclenche une 2ème requête SQL :
# SELECT * FROM courses WHERE teacher_id = 1;
print(teacher.courses)
```

B. Le Danger : Le Piège du "N+1 Select" dans une Boucle

Le Lazy Loading devient catastrophique lorsque tu parcoures une liste d'objets dans une boucle pour accéder à une propriété de navigation.

Exemple du piège :

```
# 1 requête pour récupérer 100 enseignants
teachers = session.scalars(select(Teacher)).all()

for teacher in teachers:
    # DANGER : À chaque itération, SQLAlchemy exécute 1 requête SQL supplémentaire !
    print(f"Prof: {teacher.name}, Nb cours: {len(teacher.courses)}")
```

Ce qui se passe en Base de Données :

- **1 requête** initialement pour récupérer les 100 enseignants (`SELECT * FROM teachers`).
- **100 requêtes** individuelles exécutées dans la boucle (`SELECT * FROM courses WHERE teacher_id = ...`).
- **Total = 101 requêtes SQL !** (D'où le nom **1 + N**). Si tu as 1 000 enseignants, ton application fera 1 001 requêtes BDD au lieu d'une seule, provoquant des lenteurs extrêmes.

2. L'Eager Loading : Résoudre le N+1 Select

Pour éviter ce problème, il faut utiliser le **Chargement Anticipé (Eager Loading)**. On indique à SQLAlchemy de charger immédiatement la relation **dans la même requête** (ou via une requête groupée optimisée) grâce aux fonctions passées à `.options()`.

A. `selectinload` (Recommandé pour les relations 1-N et N-N)

`selectinload` exécute **deux requêtes SQL optimisées** : la première pour charger les parents, et la seconde utilisant l'opérateur SQL `IN` pour charger tous les enfants en un seul coup.

```
from sqlalchemy.orm import selectinload

# 1. Construire la requête avec l'option selectinload
stmt = select(Teacher).options(selectinload(Teacher.courses))

teachers = session.scalars(stmt).all()

# Plus AUCUNE requête SQL n'est exécutée dans la boucle !
for teacher in teachers:
    print(f"Prof: {teacher.name}, Cours: {[c.title for c in teacher.courses]}")
```

Requêtes SQL générées sous le capot :

```
-- Requête 1 : Récupère les profs
SELECT * FROM teachers;

-- Requête 2 : Récupère TOUS les cours de TOUS ces profs en une seule fois
SELECT * FROM courses WHERE teacher_id IN (1, 2, 3, 4, ...);
```

B. `joinedload` (Recommandé pour les relations N-1 et 1-1)

`joinedload` réalise une véritable jointure SQL (`LEFT OUTER JOIN`) dans la BDD pour ramener le parent et l'enfant en **une seule et unique requête SQL**.

```
from sqlalchemy.orm import joinedload

# Récupérer les cours avec leur enseignant associé
stmt = select(Course).options(joinedload(Course.teacher))

courses = session.scalars(stmt).all()

for course in courses:
    print(f"Cours: {course.title}, Prof: {course.teacher.name}")
```

Requête SQL générée :

```
SELECT courses.id, courses.title, teachers.id, teachers.name
FROM courses
```



```
LEFT OUTER JOIN teachers ON teachers.id = courses.teacher_id;
```

Tableau Récapitulatif : `selectinload` vs `joinedload`

Technique	Mécanisme SQL	À privilégier pour...
<code>selectinload</code>	2 requêtes SQL (avec <code>IN (...)</code>)	Relations 1-N et N-N (listes d'objets). Évite les doublons de lignes renvoyés par SQL.
<code>joinedload</code>	1 seule requête SQL (<code>LEFT JOIN</code>)	Relations N-1 et 1-1 (objet unique lié).

3. Filtrer sur des Relations avec `.join()`

Attention à ne pas confondre `joinedload` et `.join()` !

- `joinedload` sert uniquement à **charger la donnée en mémoire** (pour remplir l'attribut Python de navigation). Il ne sert **pas** à filtrer.
- `.join()` sert à **filtrer ou trier** les résultats en fonction des colonnes de la table liée.

Exemple 1 : Filtrer les enseignants qui ont un cours spécifique

```
from sqlalchemy import select

# Récupérer les enseignants qui donnent un cours contenant "Python" dans le titre
stmt = (
    select(Teacher)
    .join(Teacher.courses) # Fait la jointure SQL INNER JOIN
    .where(Course.title.ilike("%python%"))
    .distinct() # Évite d'avoir des doublons d'enseignants si un prof a plusieurs
    cours Python
)

teachers = session.scalars(stmt).all()
```

Exemple 2 : Combiner `.join()` (filtrage) et `selectinload()` (chargement)

C'est un cas très fréquent : tu veux filtrer tes enseignants selon un critère sur leurs cours, **ET** charger la liste de leurs cours pour l'afficher plus tard sans refaire de requêtes.

```
stmt = (
    select(Teacher)
    .join(Teacher.courses) # 1. Permet de filtrer sur les cours
    .where(Course.title.ilike("%python%")) # 2. Condition de filtrage
    .options(selectinload(Teacher.courses)) # 3. Charge la liste des cours en
```

```
mémoire
    .distinct()
)

teachers = session.scalars(stmt).all()
```

4. Types de Jointures (**INNER**, **OUTER**)

Par défaut, `.join()` réalise un **INNER JOIN** (seuls les enregistrements qui ont une correspondance dans les deux tables sont renvoyés).

Si tu souhaites effectuer un **LEFT OUTER JOIN** (récupérer tous les éléments de la table principale, même ceux qui n'ont pas d'enfants/correspondances), ajoute `isouter=True` :

```
# Récupérer TOUS les enseignants, même ceux qui n'ont AUCUN cours attribué
stmt = (
    select(Teacher)
    .join(Teacher.courses, isouter=True) # LEFT OUTER JOIN
)
```

Synthèse du Chapitre

1. **Ne jamais laisser une propriété de navigation s'exécuter au coup par coup dans une boucle** → Danger de **N+1 Select**.
2. Pour **charger une liste (1-N / N-N)** → Utiliser `.options(selectinload(Model.relation))`.
3. Pour **charger un objet unique (N-1 / 1-1)** → Utiliser `.options(joinedload(Model.relation))`.
4. Pour **filtrer (WHERE) sur une table distante** → Utiliser `.join(Model.relation)`.

5 (suite) : Requêtes Avancées — Subqueries, CTE et CTE Récursives

Lorsque les filtres simples et les jointures ne suffisent plus (pour réaliser des agrégations complexes, manipuler des tables dérivées ou parcourir des arborescences), SQLAlchemy 2.0 propose des abstractions très proches du SQL natif : les **Subqueries** et les **Common Table Expressions (CTE)**.

1. Les Sous-requêtes (**subquery()**)

Une sous-requête est une instruction `select()` transformée en une table virtuelle réutilisable au sein d'une requête principale via la méthode `.subquery()`.

Cas d'usage : Récupérer les enseignants ayant plus de 3 cours

```
from sqlalchemy import select, func

# 1. Construire la sous-requête (compter les cours par enseignant)
subq = (
    select(
        Course.teacher_id,
        func.count(Course.id).label("courses_count")
    )
    .group_by(Course.teacher_id)
    .subquery() # Transforme la requête en sous-requête utilisable
)

# 2. Utiliser la sous-requête dans la requête principale
stmt = (
    select(Teacher, subq.c.courses_count)
    .join(subq, Teacher.id == subq.c.teacher_id)
    .where(subq.c.courses_count > 3)
)

results = session.execute(stmt).all()

for teacher, count in results:
    print(f"Prof: {teacher.name}, Nombre de cours: {count}")
```

La propriété `.c (Columns)` : Pour accéder aux colonnes d'une sous-requête ou d'une CTE dans SQLAlchemy, on utilise toujours `.c` (ex: `subq.c.courses_count` ou `subq.c.teacher_id`).

2. Les Table Expressions Communes (`cte()`)

Les **CTE** (définies avec `WITH` en SQL) sont très similaires aux sous-requêtes, mais offrent deux gros avantages :

1. **Lisibilité** : Le SQL généré place la sous-requête au début de l'instruction (`WITH name AS (...)`).
2. **Réutilisabilité** : Une même CTE peut être référencée plusieurs fois dans la requête principale.

Pour créer une CTE, il suffit d'appeler `.cte()` au lieu de `.subquery()`.

Cas d'usage : Trouver les cours dont le prix est supérieur au prix moyen

```
from sqlalchemy import select, func

# 1. Définir la CTE pour calculer le prix moyen des cours
avg_price_cte = (
    select(func.avg(Course.price).label("avg_price"))
    .cte("avg_price_cte") # Nom optionnel donné à la CTE
)
```

```
# 2. Utiliser la CTE dans la requête principale
stmt = (
    select(Course)
    .where(Course.price > select(avg_price_cte.c.avg_price))
)

courses = session.scalars(stmt).all()
```

SQL généré sous le capot :

```
WITH avg_price_cte AS (
    SELECT avg(courses.price) AS avg_price
    FROM courses
)
SELECT courses.id, courses.title, courses.price
FROM courses
WHERE courses.price > (SELECT avg_price_cte.avg_price FROM avg_price_cte);
```

3. Les CTE Récursives (`cte(recursive=True)`)

Les CTE récursives permettent de **parcourir des structures hiérarchiques ou en arbre** de profondeur inconnue (ex: organigramme d'entreprise, arborescence de catégories, fil de commentaires) en une **seule requête SQL**.

Une CTE récursive est composée de deux parties réunies par un **UNION ALL** :

1. **L'Ancre (Anchor)** : La requête de départ (ex: trouver le Manager racine).
2. **La partie Récursive** : La requête qui se joint sur la CTE elle-même pour descendre (ou remonter) l'arbre.

Cas d'usage : Reconstituer la hiérarchie complète d'un employé (Employé → Subordonnés)

Reprenons notre modèle récursif d'employé (`Employee`) :

```
from typing import Optional
from sqlalchemy import ForeignKey, String, select, func
from sqlalchemy.orm import Mapped, mapped_column, relationship

class Employee(Base):
    __tablename__ = "employees"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50))
```

```
manager_id: Mapped[Optional[int]] = mapped_column(ForeignKey("employees.id"))
```

Écriture de la CTE Réursive avec SQLAlchemy 2.0 :

```
# 1. L'Ancre : Récupérer le point de départ (ex: Le CEO / Manager racine avec id=1)
emp_alias = Employee
anchor_stmt = (
    select(
        Employee.id,
        Employee.name,
        Employee.manager_id,
        func.literal(1).label("level") # Optionnel : Calculer le niveau de
profondeur
    )
    .where(Employee.id == 1)
)

# 2. Initialiser la CTE avec recursive=True
hierarchy_cte = anchor_stmt.cte(name="employee_hierarchy", recursive=True)

# 3. La partie réursive : Joindre la table Employee avec la CTE
recursive_stmt = (
    select(
        Employee.id,
        Employee.name,
        Employee.manager_id,
        (hierarchy_cte.c.level + 1).label("level")
    )
    .join(hierarchy_cte, Employee.manager_id == hierarchy_cte.c.id)
)

# 4. Combiner l'Ancre et la partie Réursive avec union_all()
hierarchy_cte = hierarchy_cte.union_all(recursive_stmt)

# 5. Exécuter la requête finale
stmt = select(hierarchy_cte).order_by(hierarchy_cte.c.level)
results = session.execute(stmt).all()

for emp_id, name, manager_id, level in results:
    indent = "  " * (level - 1)
    print(f"{indent}- [{level}] {name} (ID: {emp_id}, Manager: {manager_id})")
```

SQL généré sous le capot :

```
WITH RECURSIVE employee_hierarchy AS (
    -- Ancre
```

```
SELECT employees.id, employees.name, employees.manager_id, 1 AS level
FROM employees
WHERE employees.id = 1

UNION ALL

-- Partie récursive
SELECT employees.id, employees.name, employees.manager_id,
employee_hierarchy.level + 1 AS level
FROM employees
JOIN employee_hierarchy ON employees.manager_id = employee_hierarchy.id
)
SELECT employee_hierarchy.id, employee_hierarchy.name,
employee_hierarchy.manager_id, employee_hierarchy.level
FROM employee_hierarchy
ORDER BY employee_hierarchy.level;
```

4. Bilan : Quand utiliser quoi ?

Outil	Méthode	Quand l'utiliser ?
Subquery	<code>.subquery()</code>	Pour des calculs intermédiaires simples ou des filtres <code>WHERE ... IN (SELECT ...)</code> .
CTE	<code>.cte()</code>	Quand une sous-requête est complexe, réutilisée plusieurs fois, ou pour rendre le SQL plus lisible.
CTE Récursive	<code>.cte(recursive=True)</code>	Pour parcourir des arbres , catégories parentes / enfants ou graphes de profondeur indéterminée en une seule requête SQL.